

#1. Introduction to Tabular ML

Instructor: **Hankook Lee** @ Efficient Learning Lab., Sungkyunkwan University

What Is Tabular Data?

- **Tabular data** is data organized into a two-dimensional structure consisting of **rows** and **columns**:
 - Each **row** (or sample, record, instance) represents a single observation or entity
 - Each **column** (or feature, attribute, variable) represents a measured property
 - A designated column may serve as the **target** (or label)

		column					target
row	ID	Age	Charge	Contract	Tenure	...	Churned
	1	34	\$75	Monthly	6		Yes
	2	52	\$45	2-year	48		No
	3	28	\$120	Monthly	2		Yes
	4	41	\$60	1-year	24		No

Characteristics of Tabular Data

- **Heterogeneous Feature Types**

- Tabular data combines **multiple feature types in a single row**
 - Each column can represent any kind of information

Type	Examples
Numerical	Age, price, temperature, number of children, ...
Categorical	Country, color, category, education level, ...
Binary	Yes/no, true/false, 0/1
Datetime	Timestamps, dates, ...
Text (free-form)	User comments, ...
Others	Image, video, ...

- Models must **handle very different statistical properties** within the input
- This is fundamentally harder than processing homogeneous features (e.g., pixels)

Characteristics of Tabular Data

- **No Inherent Spatial or Sequential Structure**

- In **images**, neighboring pixels are correlated → convolutions exploit this
- In **text**, word order matters → recurrent/attention models exploit this

- In **tabular data**, **column order is arbitrary**

- E.g., swapping the order of "age" and "salary" should not change the meaning

→ Models must be **permutation-invariant across features**

Characteristics of Tabular Data

- **Small to Medium Dataset Sizes**

- Modern **image** and **text datasets** can contain **billions of samples**
 - **Tabular** datasets often include **a few thousand rows**
 - The number of columns often ranges from dozens to a few thousand
 - Due to this **data scarcity**, models with strong inductive biases (e.g., tree ensembles) are often favored over data-hungry deep networks
- Models must be **data-efficient**
 - Models must **avoid overfitting**
 - Models must **generalize** from limited and heterogeneous samples

Characteristics of Tabular Data

- **Missing Values are Common**

- Real-world tabular data frequently contains **missing entries**
 - Sensors fail
 - Customers skip questions
 - Records come from different sources

→ Models must **handle incomplete inputs**

→ Models must be **robust to missing or unavailable features**

Characteristics of Tabular Data

- **Noisy and Imperfect Labels**

- **Labels are often noisy** because they come from real-world processes

- Human decisions can be inconsistent
 - Business rules may change over time
 - Labels may be collected after the input features

- Poorly defined labels can introduce leakage

- E.g., features measured after the outcome may reveal the target

→ Models must be **robust to noisy labels**

→ Models must **avoid leakage-prone features**

→ Evaluation must **respect temporal ordering**

Characteristics of Tabular Data

- **Class Imbalance**

- Many tabular prediction problems have **highly imbalanced** distributions
 - Fraud detection: very few fraudulent transactions
 - Disease diagnosis: rare positive cases
 - Defect detection: few defective products
 - In extreme cases, the minority class may account for less than 1% of the data
-
- Models must **avoid being biased toward the majority** class
 - Models must **detect rare but important cases**
 - Evaluation must **use appropriate metrics beyond accuracy**

Characteristics of Tabular Data

- **Domain Diversity and Limited Domain Knowledge**
 - Tabular data **appears across many domains**
 - Finance, healthcare, manufacturing, education, e-commerce
 - **Domain knowledge is often limited** or hard to encode
 - Important feature interactions may be unknown
 - Some patterns may be domain-specific or spurious
- Models must **adapt across domains**
- Models must **learn feature relationships from data**
- Models must **avoid spurious shortcuts**

Characteristics of Tabular Data

- Tabular data is everywhere
- However, **tabular data is challenging to model** because it often has:
 - Heterogeneous feature types
 - No inherent spatial or sequential structure
 - Small to medium dataset sizes
 - Missing values, noisy and imperfect labels
 - Highly imbalanced class distributions
 - Diverse domains with limited domain knowledge
- **These characteristics make tabular learning different from other domains** such as vision and language modeling

Tasks on Tabular Data

- **Prediction** (a.k.a. Supervised Learning)
 - Given a training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the goal is to **estimate an unknown target y for a new record $\mathbf{x}$**
 - Regression: $y \in \mathbb{R}$, or classification: $y \in \{1, \dots, K\}$
- Example: **Credit Scoring**
 - A credit score can be built from a set of credit features of borrowers

X					y_1	y_2
ID	OnTime	CreditUtil	Years	#Inquiries	Default?	Score
1	98%	12%	18	0	No	99
2	74%	85%	3	6	Yes	50
3	91%	35%	10	2	No	90

Tasks on Tabular Data

- **Anomaly Detection**

- **Identify rare, unusual records** that do not fit the expected pattern
- It is usually **unsupervised**
 - We have abundant "normal" data but very few or no labeled anomalies
- Heterogeneous features → simple distance metrics may not work
- Example: **Credit-Card Fraud Detection**

Which record is abnormal?

TxnID	Amount	Merchant	Country	Last
1	\$42.5	Coffee Shop	KR	3.0 hours
2	\$38.0	Grocery	KR	2.1 hours
3	\$2890.0	Electronics	RU	0.1 hours
4	\$12.0	Restaurant	KR	5.4 hours

Tasks on Tabular Data

- **Clustering**

- **Group similar records together without labels**
- The goal is to **uncover hidden structure**
 - E.g., customer groups, operational regimes, patient subtypes
- Common algorithms: k-means, GMMs, DBSCAN, hierarchical clustering
- Example: **Customer Segmentation for Marketing**
 - After clustering, three customer groups might be discovered

Customer	Avg Purchase	Visits	Online Ratio	Preferred Category
1	\$180.0	8	95%	Electronics
2	\$45.0	12	10%	Groceries
3	\$320.0	2	80%	Luxury
4	\$52.0	14	5%	Groceries

Tasks on Tabular Data

- **Table Question Answering**

- **A natural-language question and a table → return an answer**
 - This may require lookup, filtering, aggregation, or multi-step reasoning **over the table**
- This requires both natural-language understanding and tabular reasoning
 - Modern LLMs have made it feasible

- Example: **Internal Sales Analytics Assistant**

Order ID	Region	Quarter	Product	Revenue
1	Asia	Q1	A	1200
2	Asia	Q2	B	2400
3	Europe	Q1	A	800
4	Asia	Q3	A	1800
5	Europe	Q3	B	1100

(Q) What was total revenue in Asia in Q3?



(A) Asia's Q3 revenue was 1,800

Tasks on Tabular Data

- **Synthetic Data Generation**

- **Generate fake-but-realistic tabular records** that preserve the statistical properties of the original data while protecting individuals
- Uses:
 - **Sharing data** across organizations **under privacy constraints**
 - **Augmenting small datasets** to improve downstream models
- Example: **Privacy-Preserving Healthcare Data Sharing**
 - Want to share a patient dataset for research, but cannot legally release real records

Patient ID	Age	BMI	BPSystolic	Diabetes?
1	62	28.4	148	Yes
2	35	22.1	120	No
3	71	31.5	160	Yes

Tasks on Tabular Data

- **Tabular data is everywhere**

- Most of the data organizations collect, and most of the decisions they make from data, take a tabular form

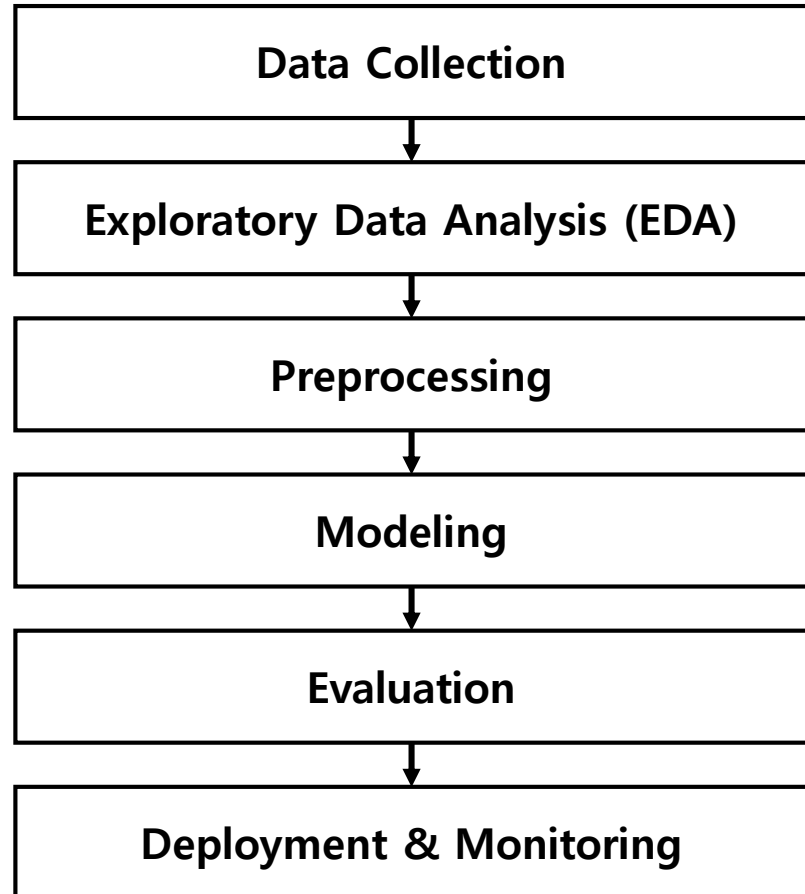
- **Many real-world problems can be expressed as tabular tasks**

- A wide range of applications across different domains and goals can be cast in the row-and-column form we have seen

(Q) How do we actually apply machine learning to a real tabular problem, from raw data to a deployed, trustworthy model?

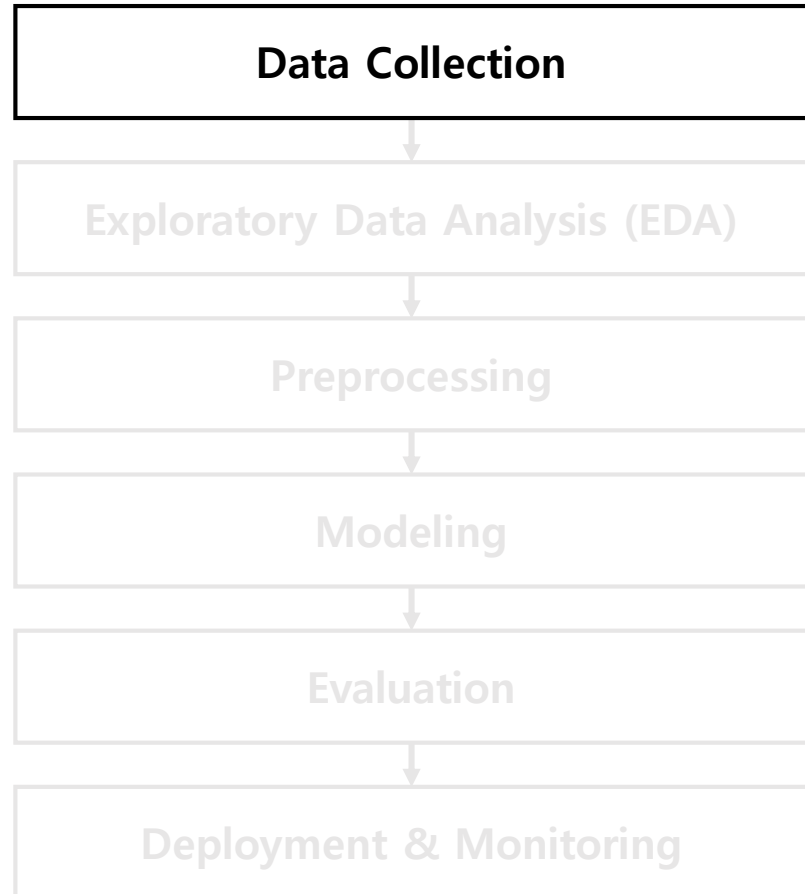
Machine Learning Pipeline

ML Pipeline



Machine Learning Pipeline

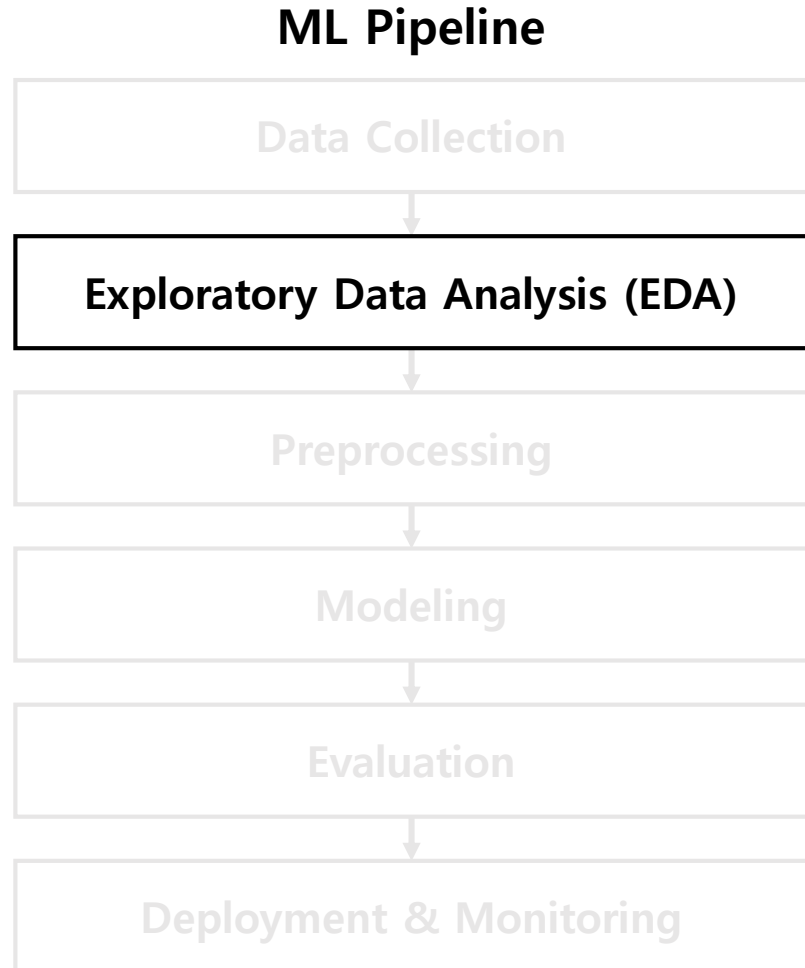
ML Pipeline



- **Data Collection**

- **Collecting relevant data from reliable sources** is the first step in an ML pipeline
- **Combine information from multiple tables**
 - E.g., customers ↔ orders ↔ products
- **Incorporate useful external data sources**
 - E.g., timestamps ↔ weather information
- Ensure **reproducibility**
 - Clearly document how, when, and from where the data was collected

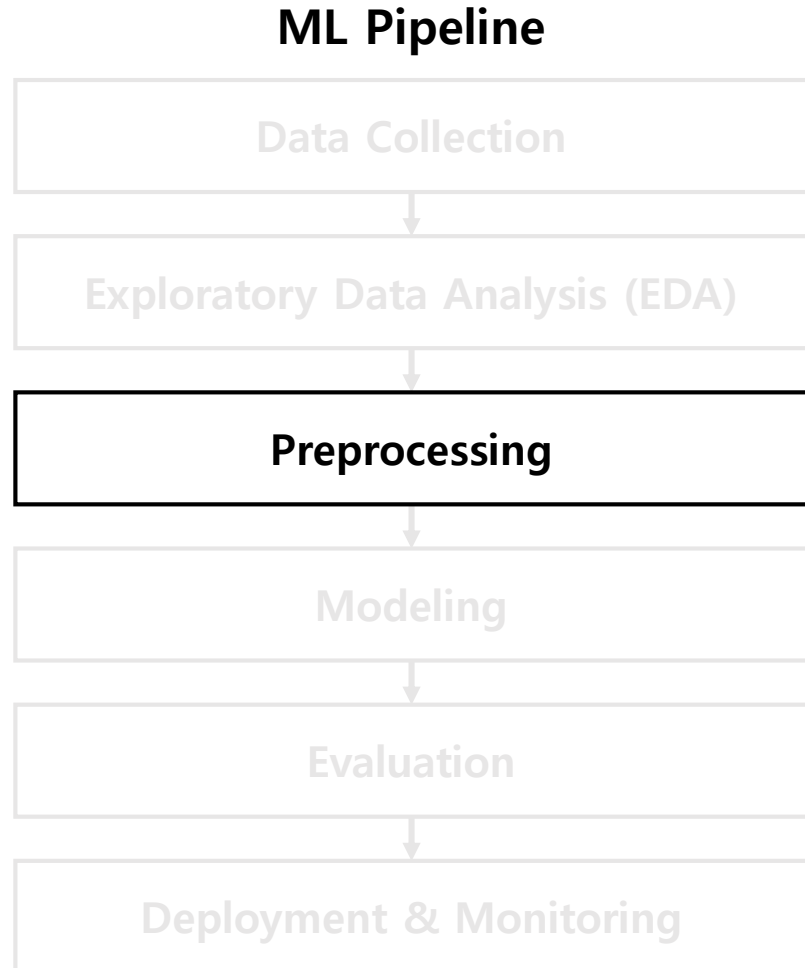
Machine Learning Pipeline



- **Exploratory Data Analysis (EDA)**

- Before modeling, **understand the data**:
 - Summary statistics (mean, median, quartiles, std)
 - Missing-value patterns (which features? how many?)
 - Distribution plots (histograms, KDE)
 - Correlation analysis between features and with the target
 - Potential outliers and class imbalance
- **EDA helps you build intuition about the data** and identify potential issues before modeling

Machine Learning Pipeline



- **Preprocessing**

- Transform raw data into a form that a model can understand/learn from
- **Preprocessing is a critical step** in building a reliable ML pipeline
 - Even a powerful model may perform poorly if the input data is not properly prepared
- **Avoid data leakage**
 - Fit preprocessing steps using the training data only
 - Apply the fitted transformations to validation/test data

Wrong

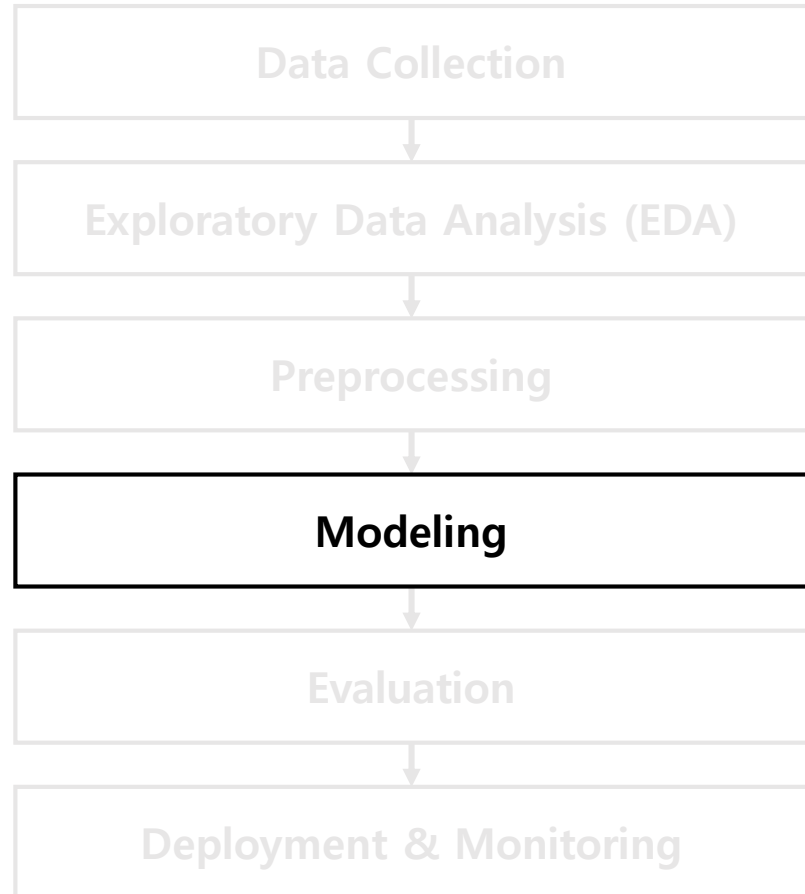
```
X = scaler.fit_transform(X)
X_train, X_test = split(X, ...)
```

Correct

```
X_train, X_test = split(X, ...)
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Machine Learning Pipeline

ML Pipeline

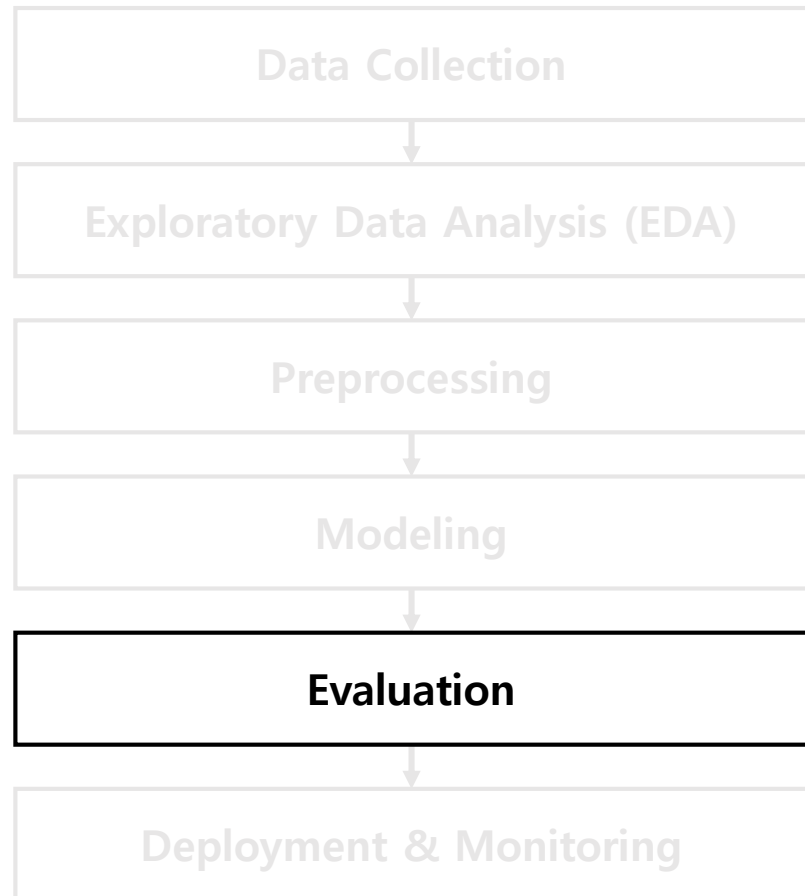


- **Modeling**

- **Choose and train a model**
- Linear/logistic regression (interpretable baselines)
- Tree-based ensembles
 - E.g., XGBoost, LightGBM, CatBoost
- Deep neural networks and foundation models
 - LLMs & TabPFNs
- Each will be studied in later lectures

Machine Learning Pipeline

ML Pipeline

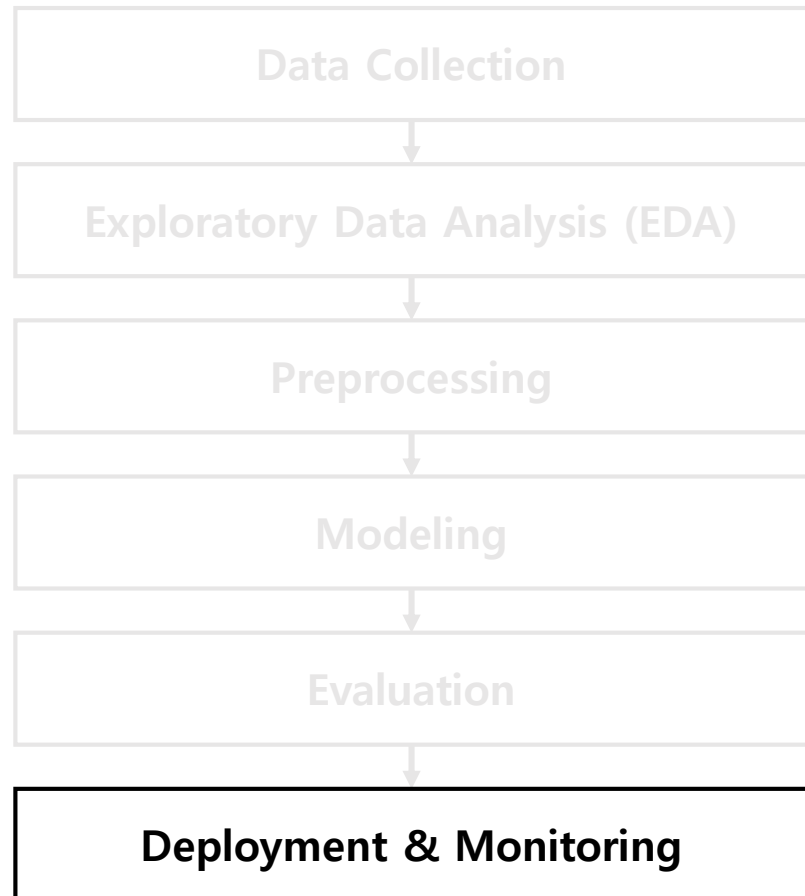


- **Evaluation**

- Use a **held-out** test set for final evaluation
- Use **cross-validation** for more reliable assessment
 - Especially useful when the available dataset is limited
- **Choose an appropriate metric** for the task
 - Imbalanced classification → accuracy may be misleading
 - Regression → MAE or RMSE
- Evaluation should reflect the real-world goal

Machine Learning Pipeline

ML Pipeline



- **Deployment**

- Consider real-world scenarios: batch vs online inference
- Ensure consistency btw training & deployment
 - E.g., the same preprocessing steps must be applied

- **Monitoring**

- Track data drift, concept drift, performance degradation, and fairness metrics
- Retrain or update the model when needed

Tabular Data Preprocessing

- Due to the heterogeneity of features, **data preprocessing plays a crucial role** in tabular ML
- You MUST consider the following before applying ML:
 - Handling missing values
 - Encoding categorical variables
 - Numerical feature transformations
 - Feature engineering

Tabular Data Preprocessing

- **Handling Missing Values**

- **Deletion:** Remove rows or columns with missing values → lose information
- **Constant fill:** Replace with "Unknown" → preserves missingness signals
- **Statistical imputation:** Replace with the mean, median, or mode
- **Model-based imputation:** Predict missing values using other features
 - E.g., use k-NN for imputation
- **Native support:** Some tree-based models handle missing values directly

Tabular Data Preprocessing

- **Encoding Categorical Variables**

- ML models require numerical input → categorical features must be encoded

Method	Basic Idea
One-Hot Encoding	Create one binary feature for each category
Ordinal Encoding	Map categories to ordered integers
Target Encoding	Replace each category with its average target value
Embedding Encoding	Learn a vector representation for each category

- **One-hot encoding** is simple and does not impose an artificial order
 - For high-cardinality features, it creates too many columns → require compact encodings
- **Ordinal encoding** is sometimes useful when categories have a natural order
 - E.g., T-shirt size: small < medium < large < x-large

Tabular Data Preprocessing

- **Numerical Feature Transformations**

- **Scaling:** Many models are sensitive to feature scale
 - **Standardization:** $x' = (x - \mu)/\sigma$ – zero-mean, unit variance
 - **Min-max scaling:** $x' = (x - \min)/(\max - \min)$ – maps to $[0,1]$
 - Some models are scale-invariant (e.g., tree-based models)
 - Scaled values may be less directly interpretable
- **Distribution transformations**
 - **Log transform:** useful for right-skewed variables (e.g., income, prices, counts)
 - **Quantile transform:** map values to their ranks in the distribution – robust to outliers
- **Binning (Discretization)**
 - This may help linear models capture non-linearities

Tabular Data Preprocessing

- **Feature Engineering**

- **Create informative features** from raw data using domain knowledge
- Well-designed features can make important patterns easier for modeling

Pattern	Example
Aggregations	Average purchase amount over the last 30 days
Ratios	Debt-to-income ratio
Time differences	Days since last login
Date decomposition	Month, day of week, or holiday indicator
Interactions	Price * Quantity
Domain-specific features	BMI in healthcare, financial indicators in finance

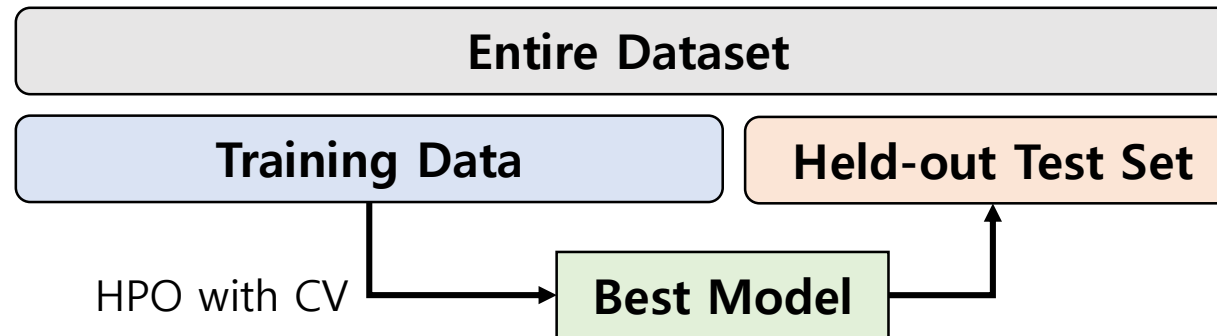
- Feature engineering can sometimes matter more than model choices

Evaluation for Tabular ML

- Tabular ML can be **highly sensitive to hyperparameter choices**
 - Tabular datasets are often relatively small and exhibit heterogeneous, irregular data patterns
- **Evaluation is essential for reliable model selection**
 - Cross-validation and hyperparameter optimization
 - Choosing the right evaluation metrics
- A strong evaluation protocol is as important as choosing a strong model

Evaluation for Tabular ML

- Cross-validation (CV) and hyperparameter optimization (HPO)
 - Use a **held-out test set** for final evaluation
 - Use **cross-validation** to obtain a more reliable validation score
 - **Tune hyperparameters** based on cross-validation performance



- Common Pitfalls
 - **Train-test contamination**: using test data during training & preprocessing
 - **Overfitting to one validation set**: tuning thousands of hyperparameter combinations on a single split

Evaluation for Tabular ML

- Evaluation metrics

- **Regression:** MAE, MSE, R^2 score = $1 - (\sum_i (y_i - \hat{y}_i)^2) / (\sum_i (y_i - \bar{y})^2)$
- **Binary classification:** Accuracy, Precision, Recall, F1 Score

	Pred. Positive	Pred. Negative
Real. Positive	TP	FN
Real. Negative	FP	TN

Accuracy = $(TP + TN) / \text{total}$

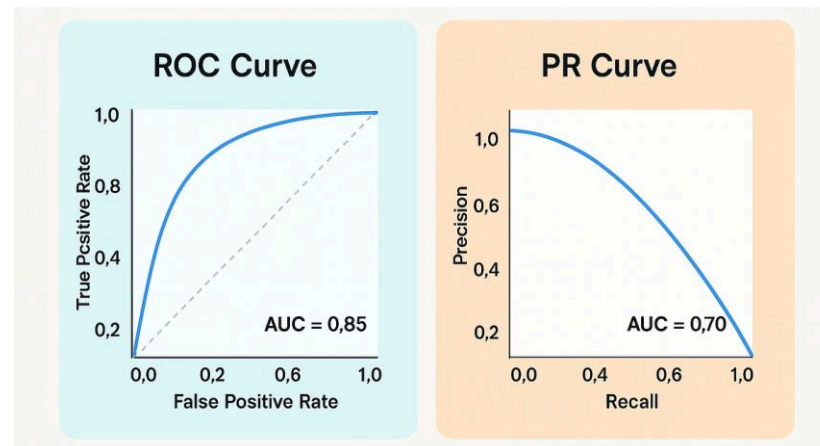
Precision = $TP / (TP + FP)$

Recall = $TP / (TP + FN)$

F1 Score = harmonic mean of precision and recall

- **Threshold-independent metrics:** AUC-ROC, AUC-PR

- AUC-PR is more informative than AUC-ROC for highly imbalanced data

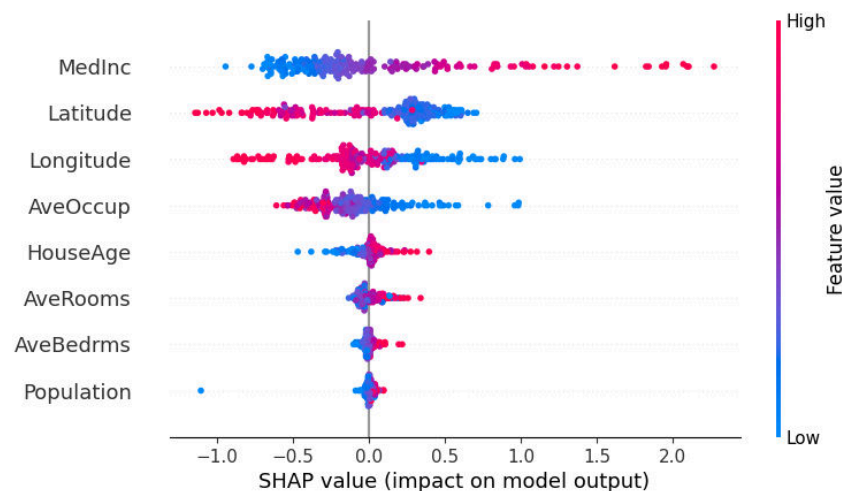


Evaluation for Tabular ML

- **Choose the right metric** for your problem
 - **Fraud detection**: very rare positives → use PR-AUC, recall at fixed precision
 - **Medical screening**: missing a disease is catastrophic → optimize recall
 - **Spam filtering**: false positives are costly → optimize precision
- The best metric is the one that reflects the real-world cost of errors

Interpretability

- We often need to **understand why a model makes a prediction**
 - High predictive performance alone is not enough
 - **Globally**, which features matter to the model overall?
 - **Locally**, why this prediction for this sample?
- **Key techniques**
 - **Permutation Importance**: Performance drop after shuffling a feature
 - **SHAP**: Feature contributions to predictions
 - Especially efficient for tree-based models



Tools & Datasets

- **Tools**

- NumPy, Pandas, scikit-learn, ...
- AutoML libraries (e.g., Optuna), Deep Learning libraries (e.g., pytorch), ...

- **Public Datasets and Benchmarks**

- Kaggle
- UCI Machine Learning Repository
- OpenML
- AI Hub (aihub.or.kr)
- ...

Summary & Lecture Roadmap

- **Tabular data is everywhere**
- Yet, **learning from tabular data is still challenging**
 - Heterogeneous features, missing values, limited data, imbalanced labels, and domain-specific patterns
- In the following lectures, we will cover:
 - Classical ML for Tabular Data
 - Deep Architectures for Tabular Data
 - Tabular Representation Learning
 - LLMs with Tabular Data
 - A New Paradigm: TabPFN

Thank You for Your Attention!
